DUT-2026大数据分析技术

Big Data Analytics



《大数据分析技术》
环境安装与课程使用

宋郅鹏，杨炘盛

Course & Assessment Overview

Dalian, October 14, 2026

这份学生手册，最终要帮助你完成什么？

- **本课程以《DataChallenging 学生操作手册》作为实际操作依据。请按照手册完成环境安装、命令执行、结果检查和阶段提交。课堂 PPT 用于说明学习路线，具体操作步骤以手册为准。**

01 安装课程环境

按照手册完成 WSL2、Ubuntu、Docker Engine 和课程离线包安装。

完成标志：`datac doctor` 检查通过

02 完成 Phase 1

配置工作区，设计 Task v1、Agent v1，运行 Reference 和 Agent。

完成标志：Build、Validate 并提交 P1

03 完成 10-Task Exposure

导入教师发放的 10 个匿名 Task，使用 Agent v1 测试，分析成功与失败原因。

完成标志：10 个 Task 运行并形成诊断记录

04 完成 Phase 2

根据测试证据完善 Task v2、Agent v2，完成 Reviews 和实验报告。

完成标志：Build、Validate 并提交 P2

课程操作路线总览图

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

阶段 ① 安装课程环境

WSL2 + Ubuntu → Docker → BigData → API 配置

完成标准：doctor 检查通过，Agent 可正常连接课程 LLM

阶段 ② 完成 Phase 1

填写 Metadata → 设计 Task v1 / Agent v1 → Reference → Agent Run

完成标准：Build → Validate → 提交 P1

阶段 ③ 完成 Exposure

导入 10 个匿名 Task → 使用 Agent v1 运行 → 分析成功与失败

完成标准：10 个任务有运行结果，完成 Review 记录

阶段 ④ 完成 Phase 2

改进 Task v2 / Agent v2 → 完成实验报告 → 测试最终版本

完成标准：Build → Validate → 提交 P2

学生手册怎么使用?

第一次安装： 按 WSL2 → Docker → BigData → API 配置的顺序操作。

课程实践： 按 Phase 1 → Exposure → Phase 2 查阅对应章节。

遇到错误： 先查"常见问题"，按环境、文件、Reference、Agent 分层排查。

01 环境安装

WSL / Docker / BigData / API

doctor PASS + LLM 可用

02 Phase 1

Task v1 + Agent v1

Build / Validate / 提交 P1

03 Exposure

10 个匿名 Task + Review

运行、分析、记录

04 Phase 2

Task v2 + Agent v2 + 报告

Build / Validate / 提交 P2

第一次按顺序安装 • 日常按阶段查阅 • 出错按层排查

每完成一个阶段，先验证结果，再进入下一阶段

按照手册，独立完成环境安装、任务设计、Agent 测试、失败分析和最终提交。”

“环境安装 → Phase 1 → Exposure → Phase 2”逐步完成；能够按要求完成：安装 → 实践 → 诊断 → 提交。

DataChallenging 课程环境

- **Windows → Ubuntu → Docker → DataChallenging**
- **按照手册从第一层到第四层逐步完成课程环境安装，每装好一层，先检查是否正常，再继续下一层**
- **进入课程后，你主要使用 bin/datac 命令，在工作区完成 Task 和 Agent 的开发、测试以及提交。**

第 1 层：Windows（宿主系统）

你的电脑运行 Windows，通过 WSL2 提供 Linux 运行环境。

第 2 层：Ubuntu 22.04（Linux 环境）

在 Ubuntu 中安装 Docker Engine，由 systemd 管理 Docker 服务。

第 3 层：Docker Engine + Compose

加载课程离线镜像，创建并启动 DataChallenging 容器。

第 4 层：DataChallenging 课程系统

使用 `bin/datac` 操作课程工作区，完成 Task、Agent、P1 / Exposure / P2。

bin/datac　workspace/　phase1 / exposure / phase2

↓↑ Agent 运行时通过网络调用

外部服务：课程 LLM API

为 Agent 提供模型能力，需要按手册配置 `LLM_BASE_URL` 和 `LLM_API_KEY`，并保证网络可用。

安装系统前通过的检查

先确认这 4 件事，再开始安装

① Windows 版本符合要求

按 `Win + R`，输入 `winver`。

检查通过标准

Windows 11，或 Windows 10 2004 / Build 19041 及以上

② CPU 虚拟化已经开启

任务管理器 → 性能 → CPU。

检查通过标准

虚拟化：已启用

③ 拿到正确的课程 Release

使用教师发放的 BigData 安装包，核对版本及文件是否完整。

重点核对

`images/`、`docker-compose.yml`、`scripts/`

④ 确认使用官方安装命令

按照当前 Release 配套手册安装，优先使用提供的安装脚本。

操作要求

不自行猜版本、镜像名、容器名或命令

Windows版本不符合要求、虚拟化未开启、安装包不完整，均应先参照手册排查;无法解决时联系助教。

安装 WSL2 + Ubuntu 22.04 LTS

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

① 打开 PowerShell（管理员）

右键点击 Windows 开始菜单，选择"终端（管理员）"或"Windows PowerShell（管理员）"。

确认窗口具有管理员权限后再执行安装命令。

② 安装 Ubuntu 22.04

在管理员 PowerShell 中依次执行：

```powershell
wsl --update
wsl --install -d Ubuntu-22.04
```

若提示需要重启电脑，请先重启。

③ 首次启动 Ubuntu

打开 Ubuntu 22.04，按提示创建 Linux 用户名和密码。

密码输入时不显示字符，属于正常现象；输入完按 Enter。

④ 检查安装是否成功

回到 Windows PowerShell，执行：

```powershell
wsl -l -v
```

成功标准

看到 `Ubuntu-22.04`，且 `VERSION = 2`。

安装失败怎么办？

如果出现下载失败、虚拟化错误或 Ubuntu 无法启动，先查看学生手册“WSL2 安装与常见问题”章节。不要反复运行安装命令。

Windows PowerShell — 安装结果示例

```
PS C:\> wsl -l -v

  NAME            STATE     VERSION
* Ubuntu-22.04    Running   2
```

Ubuntu-22.04 已安装，WSL 版本为 2

配置 Ubuntu，准备 Docker 运行环境

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

① 进入 Ubuntu，确认系统版本

打开 Ubuntu 22.04 终端，执行：

Bash

```
whoami
cat /etc/os-release
```

检查结果：能显示当前用户名，系统版本为 Ubuntu 22.04。

② 检查并启用 systemd

在 Ubuntu 中执行：

Bash

```
ps -p 1 -o comm=
```

如果显示 `systemd`，直接进入第④步。否则按第③步配置。

③ 未启用时：修改配置并重启 WSL

在 Ubuntu 中编辑配置：

Bash

```
sudo nano /etc/wsl.conf
```

确保文件包含：

INI

```
[boot]
systemd=true
```

保存退出后，在 Windows PowerShell 执行 `wsl --shutdown`，重新打开 Ubuntu。

④ 安装基础工具并检查

在 Ubuntu 中执行：

Bash

```
sudo apt update
sudo apt install -y curl wget git unzip ca-certi
ps -p 1 -o comm=
```

完成标准

基础工具安装成功，最后一条命令显示 `systemd`。

Ubuntu 终端 — systemd 检查示例

```
$ ps -p 1 -o comm=
systemd

$ cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
...
```

Ubuntu 22.04 + systemd 已就绪

版本号仅为输出示例；Ubuntu 22.04 的具体补丁版本可能不同。

Ubuntu 22.04 仓库安装方式作为简化操作示例

① 打开 Ubuntu 终端

确认上一阶段的 systemd 已启用：

Bash

```
ps -p 1 -o comm=
```

应输出 `systemd`，然后继续安装。

② 安装 Docker + Compose

使用 Ubuntu 仓库安装：

Bash

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2
```

需要联网获取软件包；若提示找不到软件包，查手册中的仓库检查方法。

③ 启动 Docker 服务

让 Docker 立即运行，并在 Ubuntu 启动时自动启动：

Bash

```
sudo systemctl enable --now docker
```

④ 验证安装结果

Bash

```
sudo docker version
docker compose version
sudo docker ps
```

必须能查看 Engine 版本、Compose 版本，并正常连接 Docker 服务。

Ubuntu — Docker 安装验证示例

```
$ docker compose version
Docker Compose version v2.x.x

$ sudo docker ps
CONTAINER ID   IMAGE   COMMAND   STATUS
```

✓ Compose 已安装，Docker 服务可以正常访问

安装课程系统

① 进入课程安装包目录

打开 Ubuntu 终端，进入已经解压的 BigData Release 根目录。

Bash

```
cd /path/to/BigData-v10.1.XX
ls
```

将示例路径替换为实际安装路径。确认存在 `images/`、`scripts/` 和 `docker-compose.yml`。

② 执行一键安装

仍在 Ubuntu 终端中执行：

Bash

```
bash scripts/install.sh
```

等待安装脚本执行完成，不要中途关闭终端。

③ 安装脚本自动完成以下操作

STEP 1	STEP 2	STEP 3	STEP 4
加载离线镜像	启动课程容器	检查运行环境	显示检查结果
Docker Load	Docker Compose	datac doctor	PASS / 问题

④ 确认安装成功

脚本执行完成后，检查是否报告 `doctor PASS`、课程容器是否正常运行。如果检查失败，按照学生手册排查。

- **先进入 BigData Release 根目录，再执行 scripts/install.sh。安装脚本会自动加载课程镜像、启动容器并检查运行环境。**
- **确认目录正确：执行 ls，应看到 scripts/、images/ 和 docker-compose.yml。**

配置课程 LLM API

- **1. 进入课程目录**

```
cd /path/BigData-v10.1.XX
```

- **2. 复制配置模板，创建 .env 文件**

```
cp workspace/.env.example workspace/.env
```

- **3. 确认 .env 文件已经创建**

```
ls -la workspace/
```

- **4. 编辑 .env 文件**

```
nano workspace/.env
```

① 获取课程 API 信息

按照教师发放的学生手册“API 配置”章节，获取：

`LLM_BASE_URL`
课程 LLM 服务接口地址



`LLM_API_KEY`
用于访问 LLM 服务的密钥

② 找到工作区配置文件

进入课程工作区，找到：

```
workspace/
├── .env        ← 配置这里
├── phase1/
├── phase2/
├── exposure/
├── runs/
└── store/
```

`.env` 是隐藏文件；在文件管理器中需要开启显示隐藏文件。

③ 填写 API 连接参数

打开 `workspace/.env`，填写实际获取的信息：

workspace/.env

```
LLM_BASE_URL=<课程 API 地址>
LLM_API_KEY=<你的 API Key>
```

保存文件。尖括号中的内容只是占位符，不能原样保留。

Ubuntu 终端 — 配置课程 API

```
# 1. 进入课程目录
cd /path/BigData-v10.1.XX

# 2. 查看工作区配置文件
ls -la workspace/

# 3. 编辑 .env 文件
nano workspace/.env
```

填写 API 地址和 Key → Ctrl + O 保存 → Enter 确认 → Ctrl + X 退出

④ 检查配置与模型调用

在课程规定的 CLI 环境中执行：

Bash

```
bin/datac doctor --workspace /workspace
```

环境检查通过后，运行 Agent 验证实际 LLM 调用：

Bash

```
bin/datac run --phase phase1
```

`doctor` 通过不一定代表 API 已连接成功；需要通过实际 LLM 调用

统一使用 bin/datac 操作课程系统

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

- **第一类是 reference，用于运行标准答案程序；**
- **第二类是 run，用于测试自己编写的 Agent；**
- **第三类是 submission，用于构建和验证阶段提交包；**
- **第四类是 exposure，用于测试教师发放的匿名 Task。**

① 进入课程目录，检查 datac 是否可用

课程终端

```
# 1. 进入课程 Release 根目录
cd /path/BigData-v10.1.XX

# 2. 检查课程命令是否可用
bin/datac --help

# 3. 检查课程运行环境
bin/datac doctor --workspace /workspace
```

`--help` 正常

能够显示课程 CLI 的命令及使用说明。

`doctor` 检查通过

课程要求的基础运行环境通过检查。

② 认识后续课程常用的四类命令

`reference` — 运行标准答案

运行 `reference.py`，确认 Task 的参考计算结果。

示例：`bin/datac reference run --phase phase1`

`run` — 运行自己的 Agent

执行 Agent，查看运行情况与输出结果。

示例：`bin/datac run --phase phase1`

`submission` — 构建与验证提交包

完成 P1 / P2 的 Build 和 Validate。

`exposure` — 测试匿名 Task

导入、检查并运行教师发放的 10 个匿名任务。

课程工作区：直接修改对应阶段的文件

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

① 认识课程工作区

课程 Workspace 结构示意

```
workspace/
├── .env.example        # API 配置模板
├── .env                # 实际 API 配置（首次复制生成）
│
├── phase1/             # 第一阶段：Task v1 + Agent v1
│   ├── task.json
│   ├── data.csv
│   ├── reference.py
│   ├── agent.py
│   └── metadata.json
│
├── phase2/             # 第二阶段：Task v2 + Agent v2
│   ├── task.json
│   ├── data.csv
│   ├── reference.py
│   ├── agent.py
│   ├── metadata.json
│   ├── review-comments.json
│   └── 实验报告模板.docx
│
├── exposure/           # 教师发放的匿名 Task
├── runs/               # 系统运行结果
└── store/              # 系统缓存与构建产物
```

无需 init

② 明确不同目录的用途

Phase 1 — 初始设计

在 `phase1/` 中完成 Task v1、Agent v1，进行测试并提交 P1。

Phase 2 — 最终设计

在 `phase2/` 中完善 Task v2、Agent v2，完成 Reviews 和实验报告。

Exposure — 匿名任务测试

接收教师发放的 10 个匿名 Task，使用自己的 Agent v1 进行测试。

Runs / Store — 系统管理

保存运行记录与构建文件，不作为 Task、Agent 的主要编辑位置。

注意： 不要重复初始化工作区；匿名 Task 可以直接覆盖到 phase1/；但不要混用两个阶段的工作区。

课程终端 — 检查 Workspace

```
# 1. 进入课程 Release 根目录
cd /path/BigData-v10.1.XX

# 2. 检查课程工作区状态
bin/datac doctor --workspace /workspace

# 3. 查看工作区目录（在容器内执行）
ls -la /workspace
ls -la /workspace/phase1
ls -la /workspace/phase2
```

开始 Phase 1 前：先填写 metadata.json

- **开始 Phase 1 前：先填写 metadata.json**
- **每个人必须先填写自己的学号、Task ID、Agent ID 和预设失败模式。**
- **系统使用这些信息识别并管理你的课程提交。**

先修改 phase1/metadata.json，填写真实学号、统一格式的 Task / Agent ID 和预设 Failure Mode，再开始设计 Task 和 Agent。

① 修改 Phase 1 的配置文件

workspace/phase1/metadata.json

```
{
  "workspace": "phase1",
  "student_id": "20250001",
  "task_id": "TASK_20250001",
  "agent_id": "AGENT_20250001",
  "hypothesized_failure_mode":
    "FM_AGGREGATION"
}
```

示例学号为 20250001，实际填写时必须替换为自己的学号。

② 按统一规则填写 ID

`student_id`：自己的学号

`task_id`： `TASK_学号`

`agent_id`： `AGENT_学号`

③ 选择预设 Failure Mode

`hypothesized_failure_mode`：预测这道 Task 最可能让 Agent 出现的错误类型。

例如 `FM_AGGREGATION`（聚合计算错误）；必须从课程规定的 Failure Mode 列表中选择。

④ Phase 2 保持身份与对象 ID 一致

`student_id`、`task_id`、`agent_id` 沿用 Phase 1；将 `workspace` 改为 `phase2`，并按课程要求填写 `review-comments_id`。

提交前检查：不能保留模板学号、不能填错阶段、不能使用旧版数字 FM 编号，例如 `FM_06_AGGREGATION`。

Phase 1：完成你的 Task 和 Agent 初始设计

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

第一部分：设计 Task

3 个文件

① task.json

写清楚要解决什么问题

说明数据含义、处理条件、计算目标、输出要求和标准答案。

你的任务：让别人能够准确理解题意。

② data.csv

准备解决问题需要的数据

保证字段存在、数据格式正确，并能支持题目所要求的计算。

你的任务：保证题目能够根据数据求解。

③ reference.py

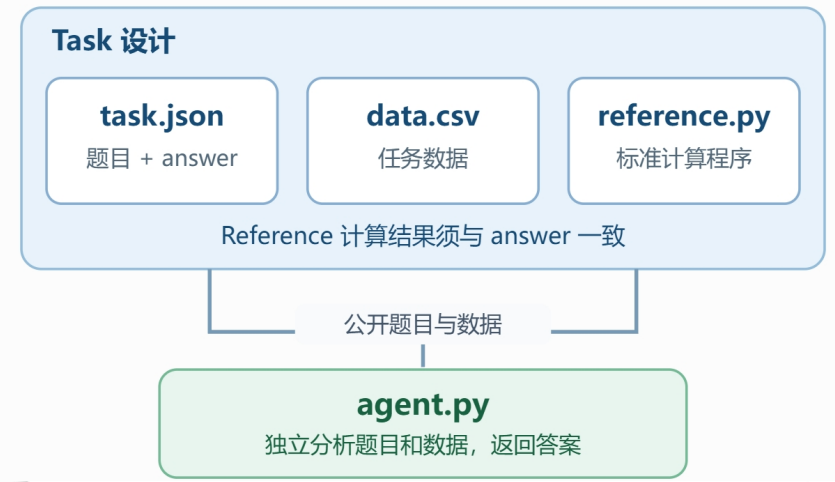
编写标准答案计算程序

根据 `data.csv` 确定性计算正确结果，不调用 LLM。

你的任务：确保计算结果与 `task.json.answer` 一致。



Task = 题目要求 + 数据 + 可验证的标准答案



第二部分：开发 Agent

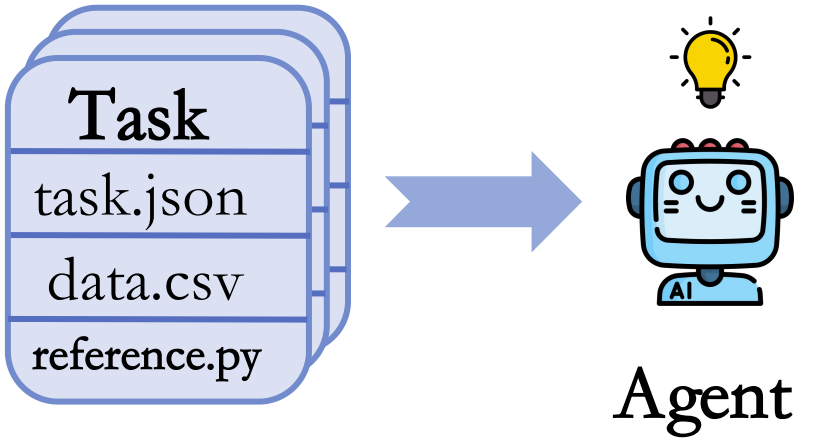
1 个文件

④ agent.py 通用数据分析智能体

编写自己的 Agent，理解题目要求，读取允许访问的数据，完成分析并返回答案。

重点： Agent 应具有解决不同 Task 的能力，而不是只针对自己设计的题目写死答案。

关键区别： `reference.py` 用来计算 Task 的 Ground Truth； `agent.py` 用来独立求解 Task。Agent 运行时不能读取 `reference.py` 或 `task.json.answer`。



Phase 1：先验证 Reference，再运行 Agent

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

01 **第一步：运行 Reference，验证标准答案**

使用 `data.csv` 执行 `reference.py`，检查计算结果是否与 `task.json.answer` 一致。

```
bin/datac reference run --phase phase1
```

- ✓ 检查通过： Reference 正常执行，Answer Check = PASS，进入第二步。
- ✗ 检查失败： 修改 `task.json`、`data.csv` 或 `reference.py`，重新运行验证。

↓ Reference 验证通过后，才能继续

02 **第二步：运行 Agent，检查求解结果**

使用自己编写的 `agent.py` 独立分析 Task 和 Data，查看运行状态及最终答案。

```
bin/datac run --phase phase1
```

根据运行结果区分三种情况：

✓ 答案正确	⚠ 答案错误	✗ 运行失败
正常执行，输出与 Ground Truth 一致	执行完成，但分析或计算结果不正确	出现程序异常、接口错误或执行中断

ⓘ 关键原则： Reference 负责验证标准答案计算链路，Agent 负责独立求解。不能为了让 Agent 得到正确结果，就让它读取或模仿 `reference.py` 的答案。



Phase 1： Build → Validate → 提交 P1

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

01

Build：生成 Phase 1 正式提交包

在课程 CLI 环境中执行，系统读取 `phase1/` 中的 Task、Agent 和 Metadata，并构建统一提交包。

```bash
bin/datac submission build \
  --input /workspace/phase1 \
  --student-id 20250001 \
  --phase P1
```

将 `20250001` 替换为自己的学号。执行完成后，记录终端输出的实际压缩包路径。

↓ 成功生成最终包后再验证

02

Validate：检查刚刚生成的压缩包

将 Build 输出的实际文件路径填入 `--input`，验证正式提交包是否符合课程规范。

```bash
bin/datac submission validate \
  --input <实际生成的P1压缩包路径> \
  --phase P1
```

确认最终包验证通过。如果验证失败，先修正工作区，重新 Build，再 Validate。

↓ Validate 通过后才提交

03

提交：上传通过验证的 P1 总包

找到最终生成的 `bigdata_p1_学号.tar.gz`，按教师指定方式提交。

不要重新压缩、解包修改或更换提交文件。

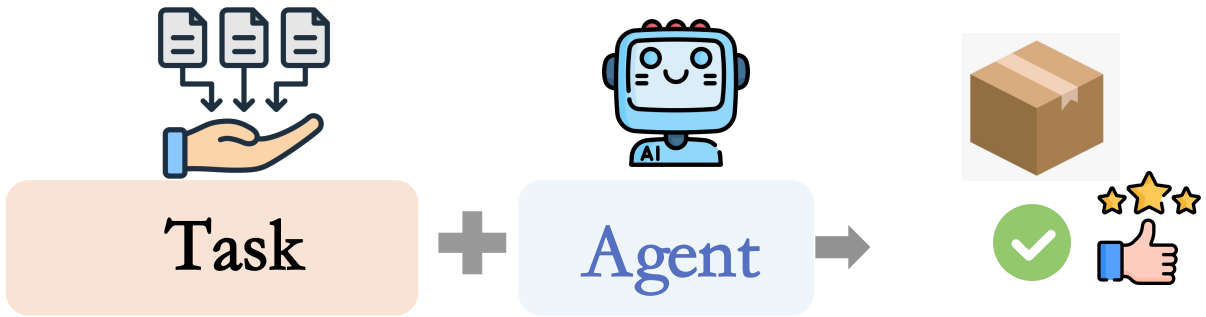
Phase 1 正式提交包

```
bigdata_p1_20250001.tar.gz
├── Tasks_20250001.tar.gz
├── Agent_20250001.tar.gz
└── metadata.json
```

最终只提交一个总包，而不是分别提交 Task、Agent 和 Metadata。

其中：

- `Tasks_学号.tar.gz`：保存你的 `task.json`、`data.csv` 和 `reference.py`。
- `Agent_学号.tar.gz`：保存你的 Agent 正式实现。
- `metadata.json`：保存阶段、学号、Task ID、Agent ID 和预设 Failure Mode 等管理信息。



bigdata_p1_{学号}.tar.gz

介绍三种运行结果

① 正确完成

程序运行成功，答案也正确

判断标准

Agent 正常执行并返回有效答案，且 Agent Output 与 Ground Truth 一致。

下一步

记录成功结果，检查输出是否稳定，继续其他测试。

② 答案错误

程序运行成功，但答案不正确

判断标准

Agent 正常返回答案，但与 Ground Truth 不一致。

下一步

检查题意理解、字段选择、筛选、聚合、计算和输出格式。

③ 运行失败

程序未能正常完成执行

判断标准

Agent 执行异常、超时、中断，或未能返回符合协议的有效输出。

下一步

检查 Python 异常、文件路径、API 连接、Runtime 和返回值。



在课程 CLI 环境中运行 Phase 1 Agent

bin/datac run --phase phase1

检查内容	确认什么
执行状态	Agent 是否正常执行，有没有异常或超时
Agent Output	Agent 实际返回了什么答案
结果比较	Agent Output 是否与 Ground Truth 一致

Phase 2 任务一：匿名 Task 测试与失败分析

① 导入匿名 Task

将教师发放的 Exposure 包导入课程工作区。

Bash

```
bin/datac exposure import \
  --package <教师发放的包路径> \
  --workspace /workspace
```

匿名 Task 进入 `exposure/`，不会覆盖自己的 Phase 1 Task。

② 检查任务是否完整

确认 10 个匿名 Task 已成功导入。

Bash

```
bin/datac exposure status \
  --workspace /workspace
```

目标：`tasks = 10 / 10`

③ 使用 Agent v1 运行

使用自己第一阶段的 Agent v1 测试全部匿名 Task。

Bash

```
bin/datac exposure run \
  --workspace /workspace
```

④ 分析结果并完成审稿

逐一查看 ATxxx 的任务要求、运行结果和可用执行证据，判断成功或失败原因。

记录到：`phase2/review-comments.json`

不仅记录是否正确，还要说明任务问题或 Agent 的失败原因。

任务一的完成目标:

10 个匿名 Task 均已运行并完成结果分析，按照课程 Review 规范记录审稿意见，为 Task v2 和 Agent v2 的改进提供依据。



Agent v1　　Agent v2

Phase 2 · 任务二：完成最终设计与提交

大连理工大学 DALIAN UNIVERSITY OF TECHNOLOGY

① 整理 10 个匿名 Task 的测试证据

回顾 Agent v1 在每个匿名 Task 上的执行情况，区分答案正确、答案错误和运行失败，分析出现问题的原因。

你需要明确：Agent v1 能解决什么问题？在哪些任务上失败？失败的证据是什么？

② 完成匿名 Task 审稿记录

按照匿名编号 `ATxxx`，记录每道任务的质量评价、发现的问题及具体依据。

保存到 `phase2/review-comments.json`

③ 完善自己的 Task v2 和 Agent v2

Task v2

检查自己 Task v1 的题目描述、CSV 数据和 Reference，必要时进行修正。

Agent v2

根据匿名任务暴露的问题，改进 Agent 的理解、分析、计算和验证能力。

没有发现需要修改的问题，可以保留原有设计，但必须在实验报告中说明原因。

④ 完成实验报告

总结 Agent v1 的测试表现、主要失败原因、采用的改进措施，以及 Task / Agent 从 v1 到 v2 的变化及依据。

重点回答：发现了什么问题？如何改进？为什么这样改？是否有证据支持？

⑤ 测试最终版本，Build → Validate → 提交 P2

在 `phase2/` 中验证最终 Task 与 Agent，完成 Build 和 Validate 后，提交唯一的 P2 总包。

最终成果	对应文件	要求
Task v2	`task.json` 、 `data.csv` 、 `reference.py`	最终任务设计与正确答案
Agent v2	`agent.py`	最终数据分析 Agent
匿名任务审稿	`review-comments.json`	完成 10 个匿名 Task 的 Review
实验报告	`实验报告模板.docx`	说明测试发现、改进过程及结果
管理信息	`metadata.json`	身份、阶段和对象 ID 等信息

Phase 2 任务二：以真实运行证据为依据，完成匿名 Task 审稿，完善 Task v2 与 Agent v2，形成实验报告，并通过 Build → Validate 提交最终 P2。

正式评测：精选测试集 + 攻防对抗

首先明确：最终使用哪些提交成果?

题目 Task（盾）	智能体 Agent（矛）
Task v1 + Task v2	仅使用 Agent v2
两道题目分别审核，最多 2 道可参评	Agent v1 不参与最终正式评测

模式一：精选测试集评测

考察 Agent 的综合解题能力

从所有有效 Task 中，综合失败模式、原创性、区分度等因素精选高质量题目。

使用 Agent v2 统一解题，成功解决的题目越多，得分越高。

题目入选测试集，可获得额外加分

模式二：攻防对抗评测

考察 Agent 进攻能力和 Task 防守能力

Agent v2 作为"矛"，挑战其他同学的有效 Task。

Task v1 / v2 作为"盾"，接受其他同学 Agent 的挑战。

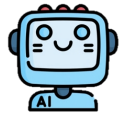
Elo Rating 分别计算进攻、防守排名和得分



看 Agent 能攻、Task 能守。



PK



Task 无歧义、标准答案正确、自己 Agent 能够成功解决，并针对规定 Failure Mode 设计；超时、资源超限及运行时错误不算有效失败模式。



参评前提：Task 与 Agent 必须符合课程规范

题目必须符合规范，没有歧义，有正确的标准答案，并且你自己的 Agent 必须能够成功解决。

最终成绩

“四项成绩汇总”的布局

① 题目质量得分

审核 Task v1 和 Task v2 的有效性、原创性与挑战性。

高质量题目入选精选测试集，可获得额外加分。

② 测试集解题得分

使用最终提交的 Agent v2，统一求解精选测试集。

成功解决的有效题目越多，得分越高。

③ Elo 进攻得分

Agent v2 作为“矛”，挑战其他同学的有效 Task。

根据攻防结果计算 Elo 进攻排名，并转换为对应分数。

④ Elo 防守得分

Task v1 / v2 作为“盾”，接受其他同学 Agent 的挑战。

根据攻防结果计算 Elo 防守排名，并转换为对应分数。

共同决定最终成绩

$$S_{\text{final}} = w_Q S_Q + w_T S_T + w_A S_A + w_D S_D$$

表示题目质量得分 **测试集解题得分** **Elo 进攻得分** **Elo 防守得分**

按照 4 个层次逐步排查

① 环境层：课程系统能否运行?

检查： Docker 容器、 `bin/datac` 命令和运行环境是否正常。

Bash

```
docker compose ps
bin/datac --help
bin/datac doctor --workspace /workspace
```

出错时： 先检查当前目录、容器状态和报错信息，按照手册修复环境。

② 文件层：工作区文件是否正确?

检查： `phase1/` 或 `phase2/` 中的文件是否完整、路径是否正确。

重点检查 `task.json` 、 `data.csv` 、 `reference.py` 、 `agent.py` 和 `metadata.json` 。

出错时： 补齐缺失文件、修正 JSON 格式、学号及阶段信息。

③ Reference 层：标准答案是否正确?

检查： Reference 是否正常执行，结果是否与 `task.json.answer` 一致。

Bash

```
bin/datac reference run --phase phase1
```

出错时： 检查 CSV 字段、数据类型、计算逻辑及标准答案。

④ Agent 层：智能体为什么失败?

检查： Agent 是否正常执行、是否成功调用 API、返回答案是否正确。

Bash

```
bin/datac run --phase phase1
```

出错时： 根据异常日志排查程序或接口；答案错误时检查题意理解、筛选、聚合和计算逻辑。

排查原则：先定位，再修改，修改后重新验证。

环境检查通过不代表答案正确；Agent 答案错误也不意味着 Docker 或 Ubuntu 有问题。

以上命令以 Phase 1 为例；Phase 2 的 Reference 和 Agent 测试将 `--phase phase1` 改为 `--phase phase2` 。Docker 检查应在能访问课程 Compose 文件的 Ubuntu 环境中执行。

01 环境层
CLI / Docker / doctor
检查环境和启动状态

02 文件层
目录 / 文件 / Metadata
检查文件与配置

03 Reference 层
执行 / Ground Truth
核对数据和参考答案

04 Agent 层
执行 / API / 答案
分析异常与求解逻辑

定位错误 → 修正对应问题 → 重新运行验证
避免为了修复单个错误而破坏已安装的环境

页面结构示意：四类问题及各自的检查方法，底部突出统一处理流程。

Thanks!

